# 01 / Four-day finish plan

24-27 October 2026 | P1-P5 integration, evidence and submission

**Goal:** finish one reliable local banking demonstration on one merged commit. Complete the remaining action-result loop, verify money-changing controls, then rehearse and package. Shared deployment and additional model providers are deferred.

## What each person has completed

| Role / estimate | Completed in the current code | Remaining before role sign-off |
|---|---|---|
| P1 - Claine **90%** | Five intents; Gemini/mock adapters; typed proposals; three clarification memories; grounded reports; safe failures; typo guards and latency shortcuts. Bounded cancel-then-transfer planner, pause/resume/stop and correlation tests. | Connect trusted execution outcomes through P2; show the next action in the browser; verify final live/deterministic demos and handoff. |
| P2 - Marvel **82%** | Real module adapters, chat/read loop, React dashboard, approval/MFA UI, Alembic upgrades, stored subscriptions and persistent simulated cancellation. Fresh policy recheck in confirm/verify. | Agent continuation; concurrent action/account protection; daily-total lifecycle; frontend metadata agreement; clean startup/build and release evidence. |
| P3 - Finance + MFA **92%** | Real analytics, report sign-off and examples. Random expiring OTP, attempt limits, demo inbox, atomic MFA claim/execution transaction, failure recovery and concurrency test cases. | Revalidate with latest security changes and PostgreSQL; browser failure/retry rehearsal; refresh stale MFA documentation and final evidence. |
| P4 - Security **87%** | Fail-closed policy and real adapter; ownership/schema/lock/tier tests; execution-time policy rechecks; changed-risk escalation to MFA and rejection reasons. | Verify concurrent money paths with P2/P3; define safety-lock behavior and daily reset; refresh security matrix to match newly merged code. |
| P5 - Subscriptions **90%** | Detector, matching/enrichment/eligibility; real backend integration; owned stored records, persistent cancellation checks and dashboard separation of evidence versus verified status. | Browser cancel/repeat/restart sign-off; planner cancellation/result validation with P1/P2; final wording and fixture evidence. |

## Meaning of these percentages

Estimates of full role completion, including integration, verification and delivery; allow approximately +/-5-10 percentage points. They are not test coverage or a guarantee that the running website matches source. Do not call any role 100% until its exit checks pass.

Review basis: local feature/agent and main at db80309, clean working tree before this document. Latest visible merges include app, MFA recovery, subscriptions and security. No remote fetch was performed; unpushed or newer remote-only work is not included. Dates are work blocks, not a claim about the competition deadline; compress Day 4 if the portal closes earlier.

# 02 / Evidence and remaining gaps

New connections are present - do not redo completed work

**Fresh verification:** 577 tests passed across agent, finance, security, subscriptions and shared tests/contracts; one existing Google SDK/Pydantic deprecation warning. No live LLM calls or bank operations were made.

**Not verified in this review:** the combined backend suite stopped during collection because the local agent virtual environment has no Alembic. The dependency is already declared in backend/requirements.txt. Frontend node_modules is absent here, so no production build was run. Fresh Docker, PostgreSQL concurrency and the live website need release checks.

| Priority | Current finding / evidence | Owner and concrete outcome |
|---|---|---|
| P0 | approvals.py returns ExecutionResult but does not call P1. chat.py saves pending_plan; the UI only updates the completed card. | P2 + P1: persist action-to-user/conversation/proposal linkage; deliver trusted outcomes; save returned state and expose the next separately approved card. |
| P0 | Yellow confirm still has a read-then-execute flow. executor.py changes balance/daily total through ORM read/modify/write; no account serialization is visible. | P2 + P4, P3 review: atomic action claim and account protection; prove one debit under duplicate and competing requests on PostgreSQL. |
| P1 | AccountModel has a cumulative daily total but no visible day/reset field. User is_locked exists, but a complete repeated-failure lock/recovery lifecycle is not evident. | P2 + P4, P3 review: explicit Shanghai-day accounting; agreed lock trigger, scope, recovery and audit. Do not lock for provider outages. |
| P1 | chat.py still inserts action_id/action_digest/tier into strict proposal.arguments for UI transport. DB payloads are canonical, but the response mixes metadata. | P2 + P1 + contract maintainer: agree compatible transport; do not alter frozen DTOs silently. Original canonical arguments must reach result correlation unchanged. |
| P1 | Some docs still say policy is not re-run at verify or list already-fixed controls as unfixed. Local backend dependencies are stale. | P3/P4 update their existing docs; P2 verifies setup requirements, migrations, default modes and builds on a clean checkout. |

## Already implemented - verify rather than rebuild

P3 finance defaults to the real engine. P5 detection/mapping and stored cancellation are connected. P4 real evaluation is available and Compose selects it by default. Confirm and verify now re-evaluate current policy. MFA claim, execution and completion share a transaction with failure recovery. Migrations replace the earlier create_all-only startup gap.

Runtime distinction: Compose defaults USE_MOCK_AGENT=false and USE_MOCK_SECURITY=false; AGENT_PROVIDER still defaults to mock. Direct local adapter defaults for agent/security remain true unless explicitly overridden. Record actual modes, not assumptions. Demo identity uses X-Demo-User-Id, not a real login system; verify submission expectations and disclose this prototype boundary.

# 03 / Four days, clear checkpoints

6 focused hours/day plus a short merge buffer | one owner per shared file

| Day | Work blocks and teammates | Exit gate |
|---|---|---|
| Day 1<br>Connect | 0:00-0:30 All: agree commit, modes, dataset and edit ownership.<br>0:30-3:00 P2+P1: outcome continuation and frontend delivery; P3 reviews MFA statuses. In parallel P4 designs concurrency/lock checks and P5 verifies current records.<br>3:00-5:00 P2+P4: action/account race fix and daily-total decision. P1 prepares deterministic continuation tests.<br>5:00-6:00: narrow PRs and targeted checks. | G1: cancellation completion reaches P1; one next transfer card appears with separate approval; original IDs/arguments and whole state are retained. Concurrency solution agreed. |
| Day 2<br>Prove safety | 0:00-1:00 All: pull merged G1 and fix dependencies.<br>1:00-3:00 P2+P3+P4: Yellow/Red, lock changes, policy escalation, retries and concurrent PostgreSQL actions.<br>3:00-5:00 P1+P2+P5: complete plan, pause/stop, failed cancellation, persistent state and replay.<br>5:00-6:00: combined suite; record exact pass/fail and remaining gaps. | G2: no debit before approval/MFA; at most one debit per action; protected balances/daily tiers; cancellation persists; rejected plan never transfers. |
| Day 3<br>Rehearse | 0:00-1:30 P2+another teammate: fresh Docker/local startup and existing-DB upgrade without resets.<br>1:30-3:30 All: browser acceptance on same commit. P1: small live Gemini smoke; P3: analytics/MFA; P4: rejection/audit; P5: subscription truth.<br>3:30-5:00 P2: focused UI readability and errors; P5 may own delegated subscription styling.<br>5:00-6:00: demo draft, screenshots and retests. | G3: another teammate starts the app from instructions; frontend build passes; critical flows have browser evidence; deterministic and live AI modes are labelled. |
| Day 4<br>Submit | 0:00-1:00 All: critical fixes only, final tests and freeze commit.<br>1:00-3:00 P1+P2 assemble brief/architecture/deck/video; P3/P4/P5 contribute checked evidence and limitations.<br>3:00-4:30: record final demo from controlled data and verify every claim.<br>4:30-6:00: confirm portal rules, open/download every artifact, submit and retain receipt/buffer. | G4: source, instructions, tests, deck/video and security assessment reference the release commit; submission is accessible and received. |

## Collaboration rule

P2 coordinates chat.py, approvals.py, executor.py, actions.py, migrations and frontend integration. P3 owns delegated MFA edits; P4 owns security semantics and review; P5 owns subscription semantics. P1 stays in agent/ and docs/agent/ unless a specific shared-file edit is delegated. Work on personal branches, use short PRs and avoid simultaneous edits to the same shared file.

If time slips: protect safe execution, required scenario evidence, reproducible startup and submission files. Defer shared hosting, another provider, extra accounts/login redesign and broad UI redesign unless the verified competition rules require them. Do not hide an unmet requirement.

# 04 / P1 - finish the agent journey

Claine | 90% now | P2 orchestration/UI, P3 report/MFA truth, P5 cancellation truth

## Already completed

Five supported intents, Gemini and deterministic mock, strict proposal/result validation, grounded balance/report replies, caller-owned transfer/spending/cancellation memory, request guards, safe diagnostics and small latency shortcuts. The new planner stores a fixed two-step dependency: cancel first, then transfer an explicit amount to a specified recipient. Pause/resume/stop and failure/replay cases are tested at agent level.

## Day 1 - pair on the missing callback

- Give P2 the exact synthetic working example from test_conversation_acceptance.py and the Bounded ordered plan section in integration.md. P2 associates each action with its owner, conversation and original proposal; confirmations/verification return trusted results through AgentAdapterService.process_turn.

- Check the envelope: conversation_id, proposal_id, tool, original arguments, status and canonical ExecutionResult data. Save the complete returned state, including pending_plan and pending_tool_requests. Do not infer completion from a button click or a user saying done.

- Help agree how the frontend receives the next reply/action without silently changing ExecutionResult. New transfer proposals must go through normal backend policy/storage and separate approval, not directly to an executor.

## Day 2 - verify the complete two-step flow

- On a disposable database: cancel an owned active subscription, confirm it, then verify exactly one transfer proposal appears. Confirm that transfer separately; check ledger, balance and audit. A cancellation itself must never increase balance.

- Test pending MFA, failed/rejected cancellation, duplicated callbacks, stop before completion, pause then cancellation completion, resume, ambiguous recipient and another conversation. No early transfer, duplicate proposal or cross-conversation memory.

## Day 3 - browser and live AI sign-off

- Rehearse balance; spending with invalid/valid month; transfer with missing and invalid amount; unknown subscription then valid selection; typo wording; never mind; provider failure. Classify connection errors separately from understanding/validation errors.

- Run a small Gemini smoke test using the model actually accepted by the key. Record observed latency and failures. Use AGENT_PROVIDER=mock for repeatable HTTP checks and label it deterministic; do not call that live Gemini.

## Day 4 - explain and hand off

Lead the short conversation/demo script and planner architecture explanation. Update existing agent instructions only where the interface/startup changed. Record supported plan wording and limits: no arbitrary DAG, no automatic rollback, no real human contact, no projected-savings transfer. Review the final video and merge P1 changes.

## Done when

All five intents work through the website; trusted approved outcomes return to the correct state; the two-step plan works with independent approvals; current reports and verified subscriptions are described accurately; deterministic tests and a labelled live smoke pass on the release commit.

Primary files: agent/service.py, planning.py, conversation.py, request_guard.py and agent/tests/. Shared connection points owned by P2: api/chat.py, approvals.py, session_store.py, db/models.py and frontend/src/App.tsx/api.ts. Your 100% does not require taking over P2 security/database/UI work.

# 05 / P2 - complete orchestration and execution

Marvel | 82% now | coordinator across P1, P3, P4 and P5

## Already completed

FastAPI/SQLAlchemy app, chat/read double-hop and caller-owned memory, real finance/security/subscription adapters, canonical stored payloads/digests, policy rechecks, MFA support, React dashboard, owned subscription records/cancellation persistence, Alembic upgrades and Docker/local startup. The remaining work is mainly shared execution and delivery.

## Day 1 - callback plus transaction boundaries

- With P1, persist or reliably retain action-to-owner/conversation/proposal linkage. After confirm/verify and terminal failures, deliver the actual correlated result to P1, save the returned state, and create any next proposal through normal policy checks. Preserve strict original arguments and retain proposal/action IDs separately.

- Agree a compatible frontend continuation mechanism. Existing confirm/verify responses are ExecutionResult; either use a separate continuation endpoint/message fetch or a coordinated versioned response change. Show the new agent reply and approval card once, including after a response retry.

- With P4, serialize claims on Yellow actions and protect the source account for competing transfers. The current ORM read/modify/write is not a proof of concurrency safety. Keep action claim, balance/daily-total change, ledger, audit and completion atomic; coordinate MFA boundaries with P3.

## Day 2 - prove current-state controls

- Test two confirmations of one action and different actions on one account against PostgreSQL. Require one debit per action, no negative balance and accurate cumulative risk. Recheck policy within the protected transaction, not against an earlier stale snapshot.

- Implement agreed Shanghai-day reset using stored day information or a validated ledger-derived total; migrate safely. With P4/P3 verify thresholds 1000.00 versus 1000.01 and two pending transfers that cross the threshold. Keep already-Red actions from being downgraded.

- Coordinate safety-lock/recovery behavior and metadata transport. UI action_id/digest/tier currently live inside proposal.arguments; choose a reviewed compatible route to keep canonical fields clean. Never silently loosen the frozen tool schema.

## Day 3 - startup and UI

- Install declared backend dependencies, run full backend/migration suites, build frontend, then test fresh Compose and old-database upgrade. Retain existing balances/cancelled state; use a disposable database for a fresh recording instead of deleting real team data.

- Make one startup path primary; verify real mode flags and model env variables. Model defaults currently still reference 3.1 Lite; document/align the chosen tested model. Polish readable cards, loading/error states and refresh behavior; delegate subscription-only UI to P5 if useful.

## Day 4 - release owner

Merge only critical fixes, tag or name the final commit, and have another teammate run it. Assemble source/README/startup commands, test/build evidence and artifact links. Confirm submission access and keep a rollback/recovery copy of the known-good demo setup.

## Done when

All real modules run through one app; result continuation works; mutations are protected under retry/concurrency; startup/migration/build pass; frontend shows accurate state and the whole demo reproduces on a teammate machine.

# 06 / P3 - finance and MFA final proof

92% now | P2 transactions/UI, P4 policy, P1 grounded explanations

## Already completed

Decimal spending analytics, timezone/month/category/anomaly rules, reports and backend finance sign-off. MFA has random six-digit codes, bounded expiry, attempt limits, owner-scoped simulated inbox, atomic challenge claim/execution/completion, failure recovery and thread-based tests. Preserve this implementation rather than redesigning it.

## Day 1 - check current integration and documentation

- Pair with P2/P1 on result statuses: pending confirmation/MFA is not completion. Check that successful, failed, rejected and cancelled outcomes reach the same conversation; no frontend callback can substitute for a backend result.

- Review P4 policy changes in confirm/verify. Refresh docs/finance/MFA.md: it still says policy is not re-run at verify, but current approvals.py calls _current_policy. Distinguish existing SQLite test evidence from unverified PostgreSQL behavior.

## Day 2 - MFA failure and concurrency gate

- With P2/P4 rerun no-debit-before-code, correct OTP exactly once, wrong code/third failure, expired action/challenge, foreign owner, newly locked account and changed daily tier. Replayed completion must return the original audit ID.

- Exercise executor refusal and unexpected failure around commit: either nothing commits and retry is safe, or a durable terminal result prevents another debit. Include a lost-response retry. Re-run simultaneous verify on PostgreSQL and check competing actions, not just duplicate verification of one action.

## Day 3 - report and browser checks

- Verify the actual selected account and requested month. Report inputs need full appropriate history and timezone-aware timestamps. Credits/refunds stay excluded, and outbound transfers count as spending. Do not mix Alice with the fixture account.

- Check browser OTP expiry/attempt messages, simulated inbox ownership, disabled completed controls and preserved audit IDs. Verify spending before/after the controlled transfer and explain the difference to P1 for recording.

| Dataset | Expected before additional demo mutations |
|---|---|
| Alice / acc_cny_001 | Fresh seeded balance CNY 15250.00; October spending CNY 80.00; September CNY 45.00. Existing database history changes these values. |
| Fixture / account_001 | October spending CNY 1843.00; no October anomalies. Refund credits are excluded, not netted. |
| P3 synthetic September | CNY 9254.70 and seven flags in the complete example. Label it synthetic; do not attribute it to Alice. |

## Day 4 - evidence and handoff

Provide the finance/MFA slide content, checked report example, known limitations and test results from the release commit. Review the recording for no premature success claim and no implication that SMS/email was sent. Simulated delivery and demo identity must remain explicit.

## Done when

Latest finance/MFA tests and integrated controls pass on the chosen database; retries cannot duplicate a debit; browser messaging and report numbers match actual data; current documentation reflects the implemented transaction/policy path.

# 07 / P4 - verify real runtime security

87% now | P2 execution/storage, P3 MFA, P1 proposal boundaries

## Already completed

Fail-closed evaluate_action, strict arguments and ownership, lock/tier boundaries, real SecurityAdapterService, bridge tests and new execution-time policy validation. Current confirm/verify check stored payload/digest, refresh policy context and can escalate a stale Yellow action to Red. Verify these improvements rather than reporting them as missing.

## Day 1 - agree precise remaining controls

- Review the Yellow action claim and source-account update with P2. Define invariants: no early debit; one execution per action; no overspend; today's cumulative total determines MFA; same action/ledger/audit linkage survives replay. Add failing tests for actual races before choosing the fix.

- Agree the safety-lock trigger, locked operations, recovery authority and audit events. Three wrong OTPs currently fail one action; that is not a user/account safety lock. Do not lock for an AI timeout. Keep read-only behavior consistent with the configured policy.

- Agree Shanghai-day reset semantics and verify the effective runtime flags. Compose selects real security, but direct local startup still needs USE_MOCK_SECURITY=false explicitly. Confirm tests invoke the actual evaluator at execution.

## Day 2 - prove the money boundary with P2/P3

- Run duplicate Yellow confirm and competing transfers against PostgreSQL; include a combined amount crossing the daily threshold and insufficient funds. Fresh policy alone is not enough if another request can change balance/total before the commit.

- Test new lock after proposal, expiry, foreign action IDs, modified digest/arguments, unknown tools and revoked subscription support. Require safe rejection plus accurate audit/reason; a valid OTP or digest must never override changed authorization.

- Review Red challenge claim and recovery tests with P3. Verify the real policy path during confirm and verify, no downgrade of prior Red, and no stale result or replay triggering another planner transfer.

## Day 3 - update evidence, then browser sign-off

- Refresh existing integration_test_matrix.md and security_self_assessment.md. Several entries refer to earlier code and list policy re-evaluation as absent. Separate implemented/automated/browser-verified/not-tested claims and identify database and mode used.

- With P1 test merchant/note injection, wrong result correlation and unapproved user text. With P5 verify candidate confidence never grants cancellation rights. With P2 show one real rejection and one audit trail in the browser.

## Day 4 - security submission review

Own the final security self-assessment. State demo identity-header limitations, visible simulated OTP delivery, no real banking/merchant integration, and any remaining concurrency/lock gaps. Do not claim arbitrary generated-code sandboxing or automatic rollback: neither follows from typed tool proposals.

## Done when

Security claims match the merged runtime and selected mode; boundary/race/lock/replay tests pass at the relevant layer; audit reasons are accurate; remaining limits are named and not presented as production guarantees.

Primary files: security/ and existing security docs/tests. P2 coordinates edits to approvals.py, actions.py, executor.py and migrations; P3 coordinates mfa.py. Existing pure policy success does not replace execution-path proof.

# 08 / P5 - sign off subscription truth and UI

90% now | P2 persistence/API/UI, P1 conversations, P4 authorization

## Already completed

Recurring detection, cadence/price/confidence/evidence handling, pure eligibility, owner-scoped mapping/enrichment, real backend detector selection, subscription storage and persistent simulated cancellation. The dashboard now separates recurrence evidence from verified active/cancelled records. The earlier missing bridge/storage tasks are completed in source.

## Day 1 - verify the real selected-account path

- With P2 verify both dashboard and chat call real P5 code, use the same selected owned account and receive sufficient historical transactions. Confirm explicit unknown/foreign account IDs are rejected; never merge different accounts into a recurring pattern.

- Check exact stored IDs and name ambiguity. Candidate IDs may differ from stored subscription IDs. Unknown/unsupported/cancelled records produce no executable cancellation; confidence alone never creates an active subscription or cancellation support.

## Day 2 - cancellation and planner acceptance

- With P1/P2 run the plan on an owned active supported record. Before approval it stays active. After completion it becomes cancelled once, with linked audit; only then may the next explicit transfer be proposed. Cancellation creates no credit or refund.

- Refresh, restart and re-list. Cancelled state must persist; repeat cancellation must not execute again. Historical recurring-charge evidence may still appear, and should not be labelled an active subscription. Reject stale actions after a record becomes cancelled or support is revoked.

- Use a disposable demo database for another full cancellation rehearsal. Existing cancelled records must not be silently reactivated by startup just to make the demo look fresh.

## Day 3 - make the subscription screen clear

- Review candidate versus verified record labels, confidence/evidence, amount/cadence, unsupported/cancelled states and the cancellation button. If P2 delegates, limit frontend edits to SubscriptionsPanel.tsx and related styling; coordinate shared App.tsx/api.ts changes.

- Check empty/loading/error states and legibility. Two recurrence candidates can be legitimate evidence; do not hide one simply because only iQIYI is a verified active record. Confirm cancellation affordances are enabled only for eligible stored records.

## Day 4 - supply final evidence

Provide one short detection/mapping example and a complete cancel/repeat/restart walkthrough from the release commit. Review the subscription scene in the video and final README. Document limited historical confidence and simulated merchant cancellation; no merchant is actually contacted.

## Done when

P5 real code is reached in HTTP/UI, evidence and verified state are distinct, supported cancellation persists and repeats safely, planner cancellation results are accurate, and screenshots/video do not claim recurrence is proof of an active subscription.

Primary files: subscriptions/ and its tests; existing docs/subscriptions/. P2 owns stored records, endpoints and executors. P1 owns agent language/state. A frontend task is useful only if explicitly delegated to avoid overlapping edits.

# 09 / Release gates and submission

One merged commit | one controlled dataset | explicit prototype limits

| Gate / lead | Evidence required before calling the project finished |
|---|---|
| Modules / P2 + all | Real P1/P3/P4/P5 selected and reached through HTTP. Record backend mode flags and actual model. Mock interpretation is labelled deterministic; it is not a live-AI result. |
| Conversation / P1 + P2 | Five intents, missing/invalid fields, typo wording, task switch, state isolation and correlated trusted outcomes. Complete the bounded cancellation-then-transfer flow with two separate approvals; pause/stop/replay safe. |
| Execution / P2 + P3/P4 | No pre-approval/MFA debit; fresh policy and expiry/lock respected; exactly one execution under replay/concurrency; protected balance and daily accounting; consistent audit. State any unresolved limitation. |
| Subscriptions / P5 + P2 | Real detection and verified stored mapping; durable cancelled status after restart; unsupported/foreign/stale requests blocked; no balance credit or automatic projected-savings transfer. |
| Startup / P2 + verifier | Declared dependencies installed; frontend production build; fresh local/Compose startup; older database upgrade preserves financial/subscription state. Another teammate reproduces from instructions. |
| Delivery / all | Release commit, accurate README, architecture/algorithm description, tests/evidence, security self-assessment, brief/deck/video and accessible submission. Recheck original brief and portal limits before upload. |

## What to record in the demo

Show spending clarification/report, real recurring evidence versus verified cancellation, transfer approval and Red simulated OTP, then the bounded ordered plan with independent approvals and one interruption/rejection. Show audit linkage and explain prototype limits. Keep the previously planned brief/deck/video limits as a checklist, but verify them against the original competition brief and current portal before final export.

## Run the full Python regression after installing dependencies

From the repo root in a separate test terminal. Do not point these checks at the persistent demo database, and do not reuse this DATABASE_URL when starting the website.

```
$env:PYTHONDONTWRITEBYTECODE = '1'

$env:PYTHONPATH = (Get-Location).Path + ';backend'

$env:DATABASE_URL = 'sqlite:///:memory:'

agent/.venv/Scripts/python.exe -m pytest -p no:cacheprovider agent/tests finance/tests security/tests subscriptions/tests backend/tests tests -q --tb=short
```

Current review: **577 passed outside backend**; backend collection blocked by missing local Alembic. Install the project's declared requirements in the selected environment, then rerun. Migration/PostgreSQL tests may require their documented explicit database setup; skipped checks are not passes.

## Final ownership and scope

P1: conversations/planner and demo explanation. P2: app/release/startup and continuation. P3: analytics/MFA evidence. P4: security evidence and risk review. P5: subscription proof and delegated UI. All review the final recording. Shared deployment, another model, more seeded accounts or a new login system are not added by this plan; check verified rules first and disclose demo identity honestly.